

RTS

Research Tracking System

Technical Documentation

Technology Stack · Architecture · Modules · Security

APPLICATION	RTS — Research Tracking System (v2)
OWNER	Qatar University
DOCUMENT	Comprehensive Technical Documentation
FRAMEWORK	Laravel 8 · PHP 8 · MySQL
CLASSIFICATION	Internal / Technical Reference

Table of Contents

RTS (Research Tracking System) — Technical Documentation

Table of Contents

1. Overview

2. Technology Stack

Backend

Frontend

Key PHP Packages (`composer.json`)

Node Dependencies (`package.json`)

3. Overall Architecture

Request Flow

Key Architectural Traits

4. Modules & Components

4.1 Dashboard & Role Switching

4.2 Grants (Grant Types)

4.3 Programs (Research Calls)

4.4 Projects (Core)

4.5 Progress Reports

4.6 Grading

4.7 Users & Profiles

4.8 Announcements

4.9 Email & Notifications

4.10 Reports

4.11 Configuration Modules

4.12 File Explorer

4.13 Budget Utilization

4.14 Admin Upload

4.15 Help / About / Team

5. Authentication & Authorization

5.1 Authentication

5.2 Roles & Permissions

5.3 Role Switching

6. API Calls & External Integrations

6.1 Student Information System (SIS)

6.2 CrossRef API (DOI verification)

6.3 Elsevier / Scopus API (backup DOI verification)

6.4 Budget API

6.5 Test Endpoint

7. Storage & File Management

- 7.1 Filesystem Configuration
- 7.2 Directory Structure
- 7.3 Deterministic Path & Naming
- 7.4 Versioning / Resubmission Logic (`ProgressController::uploadFile`)
- 7.5 File Serving (`GET /serveFile2`)
- 7.6 ZIP Downloads (`FileExplorerController`)
- 7.7 Report Templates

8. Database Schema

- Core Tables
- Key Enum Values

9. Security Aspects

- 9.1 Implemented Controls
- 9.2 Risks & Recommendations

10. Project Workflow / Business Logic

- 10.1 Workflow Stages (Single-Reviewer)
- 10.2 Additional Statuses
- 10.3 Derived Status Helpers (`Project`)
- 10.4 Deadlines

11. Grading Engine

- 11.1 Score Configuration
- 11.2 `GradingCalculator` (app/Services/GradingCalculator.php)
- 11.3 Grading Records
- 11.4 Reviewer Performance Ratings

12. Frontend Architecture

13. Configuration

14. Development & Deployment

- 14.1 Local Setup
- 14.2 Common Commands
- 14.3 Live Deployment (XAMPP + Virtual Host)
- 14.4 Testing

RTS (Research Tracking System) — Technical Documentation

Version: 2.x

Platform: Laravel (PHP)

Application: Research Project Tracking, Grading & Reporting System

Table of Contents

1. Overview
 2. Technology Stack
 3. Overall Architecture
 4. Modules & Components
 5. Authentication & Authorization
 6. API Calls & External Integrations
 7. Storage & File Management
 8. Database Schema
 9. Security Aspects
 10. Project Workflow / Business Logic
 11. Grading Engine
 12. Frontend Architecture
 13. Configuration
 14. Development & Deployment
-

1. Overview

The **Research Tracking System (RTS)** is a web-based platform that manages the full lifecycle of funded research projects, from import/registration through progress reporting, review, final grading, and reporting. It serves three primary roles:

- **Admin** — manages programs, grants, users, reviewer assignment, and aggregation.
- **LPI** (Lead Principal Investigator) — registers projects, uploads progress/final reports, and logs outcomes (publications, students, IP).
- **Reviewer** — accepts/rejects proposals, grades progress and final reports.

The system integrates with external university services (Student Information System, CrossRef, Elsevier/Scopus, Budget API) to automatically verify research outputs and student records.

2. Technology Stack

Backend

Component	Technology
Framework	Laravel 8.75 (PHP <code>^7.3 \ ^8.0</code>)
Language	PHP 8
Database	MySQL (<code>lrts_mcp</code>) via Eloquent ORM
Auth	Laravel native session auth + Laravel Sanctum (API tokens)
Templating	Blade

Frontend

Component	Technology
CSS framework	Bootstrap 5.1 + AdminLTE 3 (custom "QU Fluent" theme)
Build tool	Laravel Mix 6 (Webpack 5, SASS)
Icons	Font Awesome
Scripts	jQuery, vanilla JS, Axios
Charts	Custom CSS gauges/pills (no heavy chart lib)

Key PHP Packages (`composer.json`)

Package	Purpose
<code>maatwebsite/excel</code> + <code>phpoffice/phpspreadsheet</code>	Excel import/export of project data
<code>phpoffice/phpword</code>	Word document templates for reports
<code>barryvdh/laravel-dompdf</code>	PDF generation (report cards)
<code>guzzlehttp/guzzle</code>	Outbound HTTP calls (CrossRef, SIS, Elsevier, Budget)
<code>jeroennoten/laravel-adminlte</code>	Admin dashboard UI components
<code>yajra/laravel-datatables-oracle</code>	Server-side data tables
<code>laravel/sanctum</code>	API token authentication
<code>spatie/simple-excel</code>	Lightweight Excel reading
<code>aacotroneo/laravel-saml2</code>	(Installed) SAML SSO support

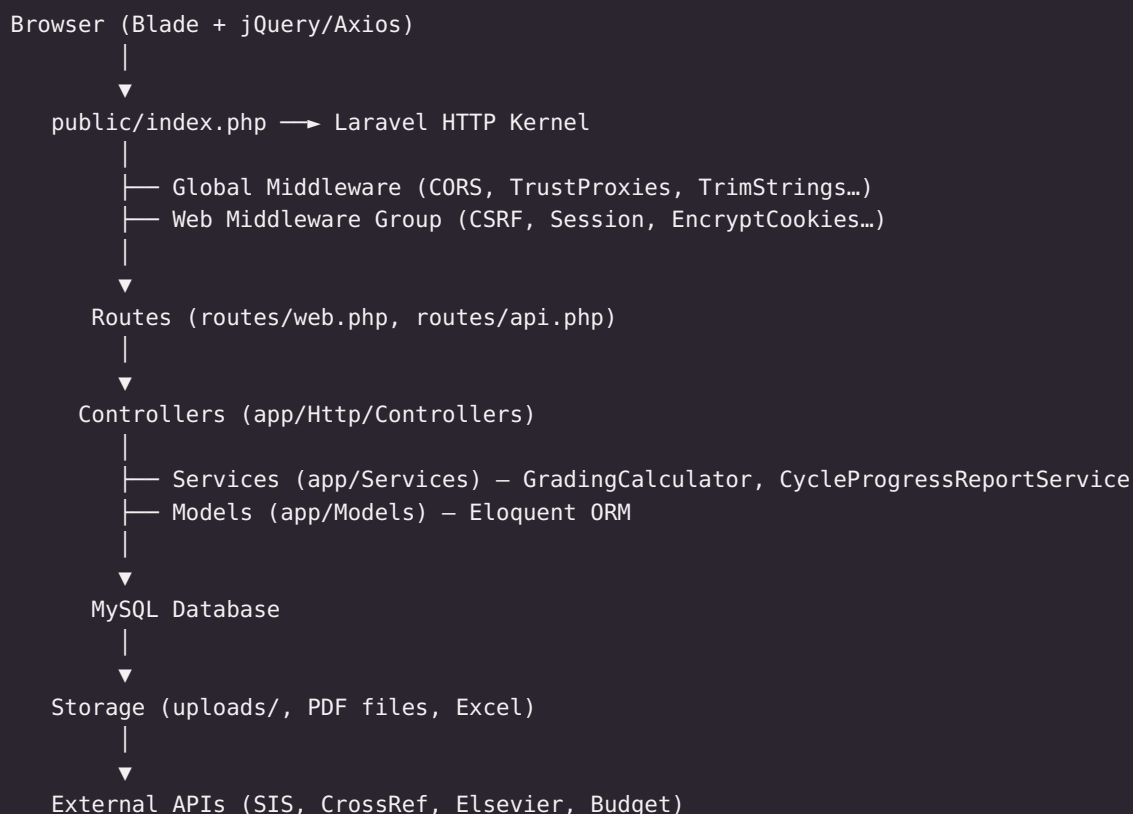
Package	Purpose
webklex/laravel-imap , sendgrid/sendgrid , phpmailer/phpmailer	Email sending
phpseclib/phpseclib	Secure communications/crypto

Node Dependencies (`package.json`)

Laravel Mix, Webpack, Bootstrap 5, Axios, Popper, Lodash, SASS.

3. Overall Architecture

The application follows Laravel's standard **MVC** pattern with a **monolithic** deployment model (single web application serving Blade views and AJAX endpoints).



Request Flow

1. User authenticates via session (CSRF-protected forms).
2. Route matched in `routes/web.php`.
3. Controller loads data through Eloquent models.
4. Blade view rendered with inline styles + shared layout (`layouts/app.blade.php`).
5. Many actions are AJAX (jQuery) hitting POST/GET endpoints returning JSON.

Key Architectural Traits

- **Role-based routing** to separate dashboards (`dashboard.admin` , `dashboard.lpi` , `dashboard.reviewer`) via `HomeController@index` .
- **Status-history driven workflow** — project progression is tracked in the `status_histories` table, not a mutable status column (though a fallback column exists).
- **Deterministic file naming** — files are stored under a hierarchical path derived from cycle year + grant code, with versioned filenames for resubmissions.
- **Server-side enforcement** of business rules (e.g., upload deadlines/locks are checked in the controller, not only the UI).

4. Modules & Components

The system is organized into the following functional modules. Each maps to one or more controllers under `app/Http/Controllers/` .

4.1 Dashboard & Role Switching

- **Controller:** `HomeController`
- **Routes:** `GET /home` , `POST /switch-role` , `GET /notifications`
- **Views:** `resources/views/dashboard/{admin,lpi,reviewer}.blade.php`
- **Behavior:** Renders a role-specific dashboard (Admin aggregates all projects; LPI sees own projects + available slots; Reviewer sees assigned work + performance ratings). Users with combined roles (e.g., `Admin+LPI+Reviewer`) can switch via session `active_role` .

4.2 Grants (Grant Types)

- **Controller:** `GrantController`
- **Routes:** `grant-types.*`
- **Models:** `Grant`
- **Purpose:** CRUD for grant types (student vs regular), grant codes, funding agency.

4.3 Programs (Research Calls)

- **Controller:** `ProgramController`
- **Routes:** `programs.*`
- **Models:** `Program` , `CycleConfig`
- **Purpose:** Manage research-call programs tied to a grant + cycle, with report deadlines. Also handles **Excel import** of project proposals and single-proposal upload.

4.4 Projects (Core)

- **Controller:** `ProjectController` , `RegisterWizardController`

- **Routes:** `projects.*`, `wizard.*`
- **Models:** `Project`, `Commitment`, `Outcome`, `ProjectPublication`, `ProjectStudent`, `ProjectResearcher`, `ProjectContribution`, `ProjectStudentDetail`, `ProjectIntellectualProperty`, `ProjectSubmission`
- **Purpose:** Core project lifecycle — listing, registration, progress/final report management, report card generation, reviewer assignment, detail view.

4.5 Progress Reports

- **Controller:** `ProgressController`
- **Routes:** `progress.*`
- **Models:** `ProjectSubmission`, `Outcome`, `ProjectStudent`, `ProjectResearcher`
- **Purpose:** LPI uploads progress/progress2/readiness/final reports (PDF), adds outcomes (articles, IP, students, researchers, contributions), verifies student details via SIS, and verifies publication DOIs via CrossRef.

4.6 Grading

- **Controller:** `GradingController`, `WorkflowController`, `ReviewerGradingController`
- **Routes:** `grading.*`, `projects.grading`, `workflow.*`, `reviewer-grading.*`
- **Models:** `ProgressReportGrading`, `FinalReportGrading`, `Score`, `Rating`
- **Purpose:** Reviewer grades progress & final reports; auto-calculation of scores via `GradingCalculator`; admin reviews of rejections; reviewer performance ratings.

4.7 Users & Profiles

- **Controller:** `UserController`, `ProfileController`
- **Routes:** `users.*`, `profile.*`
- **Model:** `User`, `Nationality`
- **Purpose:** User CRUD, role management, profile editing.

4.8 Announcements

- **Controller:** `AnnouncementController`
- **Routes:** `announcements.*`
- **Model:** `Announcement`
- **Purpose:** Role-scoped announcements shown on dashboards.

4.9 Email & Notifications

- **Controller:** `EmailController`, `EmailTemplateController`
- **Routes:** `admin.*`, `email-templates.*`
- **Models:** `EmailSendLog`, `EmailTemplate`
- **Purpose:** Compose/send/retry emails with a send log; reusable email templates.

4.10 Reports

- **Controller:** `ReportController`
- **Routes:** `reports.*`
- **Models:** `Project`, `Program`, `Grant`, `Pillar`, `CycleConfig`, `ProjectBudget`
- **Purpose:** Program status, grant summary, project status, pillar summary, cycle progress (with reminder emails), budget utilization reports (CSV/HTML).

4.11 Configuration Modules

- **Controllers:** `ScoreController`, `PillarController`, `CollegeController`, `CycleConfigController`, `GaugeSettingsController`, `TagController`, `TeamController`
- **Purpose:** Maintain lookup/configuration data (scores, pillars, colleges, cycles, gauge settings, teams).

4.12 File Explorer

- **Controller:** `FileExplorerController`
- **Routes:** `file-explorer.*`
- **Purpose:** Browse and **ZIP-download** a project's or program's report files.

4.13 Budget Utilization

- **Controller:** `BudgetUtilizationController`
- **Routes:** `budget-utilization.*`
- **Model:** `ProjectBudget`
- **Purpose:** Sync project budget data from the external Budget API and send reminders.

4.14 Admin Upload

- **Controller:** `AdminUploadController`
- **Routes:** `admin-upload.*`
- **Purpose:** Batch upload of report files by admin.

4.15 Help / About / Team

- **Controller:** `AboutController`
- **Routes:** `about.*`
- **Purpose:** Static informational pages.

5. Authentication & Authorization

5.1 Authentication

- **Mechanism:** Laravel's built-in session authentication via `Auth::routes()`.

- **Controllers:** `app/Http/Controllers/Auth/*` (Login, Register, Forgot/Reset Password, Verification, Confirm Password).
- **Login Controller:** `LoginController` uses `AuthenticatesUsers` trait; redirects to `/home` on success.
- **API tokens:** Laravel Sanctum installed; `routes/api.php` exposes `GET /api/user` under `auth:sanctum`.
- **Guest guard:** `guest` middleware (`RedirectIfAuthenticated`) prevents logged-in users from viewing login pages.

5.2 Roles & Permissions

Roles are stored as a **single string** in the `users.type` column, using a `+` separator for combined roles:

type value	Roles granted
Admin	Admin
LPI	LPI
Reviewer	Reviewer
LPI+Reviewer	LPI + Reviewer
Admin+LPI+Reviewer	Admin + LPI + Reviewer

Role helper methods on User :

- `isAdmin()` — type is `Admin` or `Admin+LPI+Reviewer`
- `isLPI()` — LPI, LPI+Reviewer, Admin+LPI+Reviewer
- `isReviewer()` — Reviewer, LPI+Reviewer, Admin+LPI+Reviewer
- `subRoles()` — splits the type string into an array
- `activeRole()` — returns the current session role (`session('active_role')`) if valid, otherwise the first sub-role

Authorization model:

- There is **no dedicated roles/permissions package** (no `spatie/laravel-permission`).
- Authorization is implemented via **inline role checks** in controllers and views, e.g. `$project->availableActions(auth()->user())` , `$this->authorizeLpi($project)` , `@if(!$user->isReviewer())` , and `$user->activeRole()` .
- The `Controller` base class / `ProgressController` has private helpers like `authorizeLpi()` and `enforceDataOpen()` to restrict LPI actions to their own projects and enforce data-open windows.

5.3 Role Switching

- `POST /switch-role` stores the chosen role in the session (`active_role`).
- `User::canSwitchRole()` returns true when a user has multiple sub-roles.

6. API Calls & External Integrations

Outbound HTTP calls are made with **GuzzleHTTP** from controllers/models. All external credentials are defined in `config/services.php` (reading from `.env`).

6.1 Student Information System (SIS)

- **Purpose:** Verify student records added to a project.
- **Model:** `ProjectStudentDetail::fetchFromApi($studentId)`
- **Endpoint:** `STUDENT_API_URL`
- **Auth:** custom headers `sec_key` and `st_id`.
- **Test mode:** `STUDENT_API_USE_TEST_RESPONSE=true` returns a canned response (`getTestResponse()`) to avoid hitting production during development. Falls back to the test response on any API error.

6.2 CrossRef API (DOI verification)

- **Purpose:** Auto-fetch publication metadata (title, journal, authors, year) from a DOI, populating `project_publications`.
- **Controller:** `ProgressController::fetchPublicationFromApi()`
- **Endpoint:** `CROSSREF_API_URL` (`https://api.crossref.org/works/{doi}`)
- **Validation:** DOI must match `/^10\.\d{4,}\./+.`
- **SSL:** `CROSSREF_API_VERIFY_SSL=false` (disabled in this environment).

6.3 Elsevier / Scopus API (backup DOI verification)

- **Purpose:** Backup source for verifying publication DOIs.
- **Configured** in `config/services.php` with `key` and `inst_token`.

6.4 Budget API

- **Purpose:** Fetch project budget utilization data.
- **Controller:** `BudgetUtilizationController::sync()`
- **Endpoint:** `BUDGET_API_URL`
- **Test mode:** `BUDGET_API MOCK=true`.

6.5 Test Endpoint

- `GET /test-crossref-api/{doi}` — a manual test route that hits CrossRef and returns the JSON result.

Note: Many external calls set `'verify' => false` on the Guzzle client, disabling SSL certificate verification. This is a **security concern** for production (see Section 9).

7. Storage & File Management

7.1 Filesystem Configuration

- **Default disk:** `local` (root = `storage/app`) via `FILESYSTEM_DRIVER=local`.
- Also configured: `public` disk (`storage/app/public`), and `s3` (unused/empty).

7.2 Directory Structure

```
storage/app/
├── uploads/                                ← Project report files
│   ├── {cycle_year}/{grant_code}/
│   │   ├── proposals/
│   │   ├── progress_reports/
│   │   ├── readiness_reports/
│   │   └── final_reports/
├── downloads/                            ← Report templates (progress/final/readiness .docx)
├── temp/                                ← Temporary uploads (Excel imports)
├── app/
│   └── email-attachments/                ← Email attachments
```

7.3 Deterministic Path & Naming

- **Directory:** `Project::getStorageDir($typeFolder) → uploads/{cycle_year}/{grant_code}/{type_folder}`.
- `cycle_year` from `program.cycle.year`
- `grant_code` from `program.grant.grant_code`
- **Filename:** `Project::getStorageFilename($type, $version) → {old_project_id}_{type}[_v{version}].pdf`
- `old_project_id` is sanitized of `/` characters.
- `type` ∈ `proposal | progress | progress2 | readiness | final`.

7.4 Versioning / Resubmission Logic (`ProgressController::uploadFile`)

Files are stored as single-file-per-type (replace on re-upload), with special **v2 resubmission** behavior:

- **Progress / Progress2 / Final** on admin resubmission after rejection: keep v1, write/overwrite **v2** (`_v2.pdf`).
- **Readiness / Final / Progress2** normal: replace existing single file (`v1`).
- Validation: `required|file|mimes:pdf|max:10240` (PDF only, max 10 MB).
- Server-side lock: LPI cannot upload once the report window is closed (`isTypeLocked()` → HTTP 423).

7.5 File Serving (`GET /serveFile2`)

- Serves stored PDFs in an iframe.
- If `submission_id` is provided, serves that exact submission version.
- Otherwise resolves the deterministic path by `type` + project `id`.

- Includes a friendly "File does not exist" placeholder instead of a 404 page.
- A generic `file` query param is guarded against path traversal (`preg_match('/(\\.\\.|\\|\\\\\\\\)/') → 403`).

7.6 ZIP Downloads (`FileExplorerController`)

- `downloadProject($id)` — Zips all of a project's report files into `project_{old_id}.zip` and streams it (deleted after send).
- `downloadProgram($programId)` — Zips all files for a research call.

7.7 Report Templates

- `GET /download-template/{type}` serves `.docx` templates (`progress`, `final`, `readiness`) from `storage/downloads/`.

8. Database Schema

DB: MySQL (`lrts_mcp`). Migrations under `database/migrations/`.

Core Tables

Table	Purpose
<code>users</code>	Users (role in <code>type</code> column, college/pillars as strings)
<code>nationalities</code>	Nationality lookup
<code>grants</code>	Grant types (category: student/regular)
<code>cycle_configs</code>	Research cycles (year, title)
<code>pillars</code>	Research pillars
<code>colleges</code>	College lookup
<code>scores</code>	Score/weight configuration for outcome types
<code>programs</code>	Research calls (grant + cycle, deadlines)
<code>projects</code>	Core project records (LPI, program, budget, student-grant fields)
<code>commitments</code>	Committed research output targets (q1-q4, IP, students)
<code>projects_reviewers</code>	Pivot: project ↔ reviewer (role, proposalstatus)
<code>project_college</code> / <code>project_pillar</code>	Pivot tables
<code>project_outcomes</code>	Achieved outcomes (type, identifier, verification flags)
<code>project_publications</code>	Publication metadata (DOI, journal, title, year)

Table	Purpose
<code>project_students</code>	Student entries on a project
<code>project_students_details</code>	Student details fetched from SIS API
<code>project_researchers</code>	Hired researchers
<code>project_contributions</code>	IP/patent contributions
<code>project_submissions</code>	Report file submissions (type, version, file_path)
<code>status_histories</code>	Workflow status audit trail (project → status → user)
<code>progress_report_grading</code>	Progress report grades/ratings per reviewer
<code>final_report_grading</code>	Final report grades (A-D, total, publish)
<code>reviewer_grading</code> / <code>reviewer_ratings</code>	Reviewer performance ratings
<code>reviewer_rejections</code>	Rejection records
<code>announcement</code>	Announcements
<code>email_send_log</code> / <code>email_templates</code>	Email tracking & templates
<code>gauge_settings</code> , <code>team</code> , <code>project_budgets</code>	Config/team/budget
<code>report_reminders_sent</code>	Cycle progress reminder tracking
<code>personal_access_tokens</code>	Sanctum tokens

Key Enum Values

- `projects.status` legacy + `status_histories.status` (see Section 10).
- `project_outcomes.verification_by_system` / `verification_by_reviewer` ∈ `verified` | `not-verified` | `pending`.
- `project_submissions.type` ∈ `progress` | `progress2` | `final` | `readiness`.
- `progress_report_grading.publish` / `final_report_grading.publish` ∈ `accepted` | `rejected` | `reserved` | `pending`.

Note: There is a minor typo in the DB — `verification_*` (misspelled "verification") is used consistently across schema and code.

9. Security Aspects

9.1 Implemented Controls

- **CSRF Protection:** All web forms are protected by Laravel's `VerifyCsrfToken` middleware; `csrf_field()` / `@csrf` used in Blade.
- **Session-based auth** with encrypted cookies (`EncryptCookies`).
- **Input validation:** Controllers validate with Laravel validation rules (e.g., `mimes:pdf`, `max:10240`, `in:...`, `exists:...`).
- **Mass-assignment protection:** Eloquent `$fillable` whitelists.
- **Password hashing:** Laravel default bcrypt; passwords hidden in serialization.
- **Path traversal protection:** `serveFile2` rejects `..`, `/`, `\` in the `file` query param.
- **Role-based UI gating:** Sensitive data (outcome/student/IP details) is hidden from reviewers in views via `@if(!$isViewer)` .
- **Server-side enforcement:** Upload deadlines/locks and LPI ownership are checked in the controller (not only the UI).
- **Route protection:** Many route groups apply `auth` middleware; the base `Controller` applies `auth` in constructors (e.g., `ProjectController`, `HomeController`).

9.2 Risks & Recommendations

1. **SSL verification disabled** — Guzzle clients use `'verify' => false` for SIS, CrossRef, Elsevier, and Budget APIs. **Enable SSL verification in production.**
2. **No dedicated authorization package** — access control is inline/role-string based. Recommend centralizing with policies/middleware to reduce risk of missed checks.
3. **APP_DEBUG=true** in `.env` — must be `false` in production to avoid exposing stack traces and config.
4. **CORS wide-open** (`allowed_origins = ['*']`, `supports_credentials = false`). Restrict origins for production.
5. **Sanctum** `expiration = null` — personal access tokens never expire. Set an expiration policy if API tokens are used.
6. **Secrets in .env** — several API keys are committed in `.env` (which is git-ignored, but be careful not to commit it). Use a secret manager / environment variables in production.
7. **/test-crossref-api/{doi}** route is unauthenticated and publicly reachable — restrict it or remove it in production.
8. **File upload validation** is good (PDF-only, size-limited), but consider scanning uploaded files for malware and storing them outside the web root (already done — `storage/app/uploads` is not publicly served).
9. **Rate limiting** — `throttle:api` is applied to the API group only; web routes (login, etc.) rely on Laravel defaults. Consider throttling login.

10. Project Workflow / Business Logic

Project progression is tracked via the `status_histories` table. Each status is recorded by `Project::recordStatus($status, $metadata, $userId)`.

10.1 Workflow Stages (Single-Reviewer)

#	Status (code)	Action	Next Status	Actor
0	<code>imported</code>	Import from Excel	<code>registered</code>	System/Admin
1	<code>registered</code>	Assign Reviewer	<code>Assigned</code>	Admin
2	<code>Assigned</code>	Accept/Reject Proposal	<code>Claimed</code>	Reviewer
3	<code>progress_added</code>	Review Progress	<code>progress_reviewed</code>	Reviewer
—	<code>progress_rejected</code>	Admin Reviews	(see admin)	Admin
4	<code>progress_reviewed</code>	Add Final Report	<code>final_added</code>	LPI
5	<code>final_added</code>	Grade Final	<code>Graded</code>	Reviewer
—	<code>final_rejected</code>	Admin Reviews	(see admin)	Admin
6	<code>Graded</code>	Report Card (view)	—	All

10.2 Additional Statuses

- **Extended/second progress report:** `progress2_added`, `progress2_reviewed`, `progress2_rejected`, `progress2_rejection_reviewed`.
- **Admin rejection review:** `progress_rejection_reviewed`, `final_rejection_reviewed`.
- **Proposal rejection:** `proposal_rejected`.
- **Reviewer removal:** `reviewer_unassigned`.

10.3 Derived Status Helpers (`Project`)

- `current_status` accessor — latest status history (falls back to `status` column).
- `latestStatus()` — `hasOne` relation via `latestOfMany()`.
- `hasStatus($status)` — whether the project ever reached a status.
- `hasUnreviewedRejection($type)` — round-aware check for whether the most recent rejection is still unreviewed (so a second/third rejection after resubmission reappears for the admin).
- `getLifecycleAttribute()` — ordered lifecycle stages with done/dates/deadlines.
- `getLifecycleStageAttribute()` — 0-based index of the current stage.
- `availableActions($user)` — computes the action menu per role + status.

10.4 Deadlines

Programs define report deadlines (`prog_rpt_deadline` , `final_rpt_deadline` , plus extended variants). The lifecycle and upload lock logic compare against `now()` to determine if a stage is "current" or closed.

11. Grading Engine

11.1 Score Configuration

The `scores` table maps outcome types to point values. `Score::getMap()` / `Score::getByCategory()` return these. Defaults (from `GradingCalculator`):

- Journal Q1-Q4: 8 / 6 / 4 / 3
- Conference: 2, Book: 8, Ed. Book: 6, Chapter: 4
- IP disclosure: 4, Provisional patent: 7, Open source SW: 8, Startup: 10
- Masters: 2, UG: 1, PhD: 3, Cross-college: 2

11.2 `GradingCalculator` (`app/Services/GradingCalculator.php`)

- `calculateExpectedScore($projectId)` — sums `commitment × score`.
- `calculateVerifiedScore($projectId)` — sums scores of outcomes flagged `verification_by_reviewer = 'verified'`, plus verified students.
- `calculateGradeA($projectId)` — $(\text{verified}/\text{expected}) \times 5$, rounded to 2 dp (outcome-verification grade on a 0–5 scale).
- `getOutcomeTypes()`, `getIpTypes()`, `getStudentTypes()` — return configured types.

11.3 Grading Records

- **Progress report grading** (`progress_report_grading`): ratings for achievements, publications, students, budget (each with a word label via the `ratings` lookup table and `/5` numeric), ethical approval, analysis, recommendation, `publish` (accepted/rejected/pending), `isAccepted`, `report_type` (`progress` / `progress2`).
- **Final report grading** (`final_report_grading`): four section grades (A: Achievements, B: Publications & IP, C: Student involvement, D: Impact), comments, `total`, `publish`, `isAccepted`.
- **Reviewer grading** (`GradingController`): saves progress & final grades, updates verification flags, and submits/publishes grades.
- **Report Card** (`ProjectController@reportCard`) — printable PDF-style page summarizing per-reviewer progress/final remarks and overall status.

11.4 Reviewer Performance Ratings

`ReviewerRating` (table `reviewer_ratings`) records admin evaluations of reviewers per program/cycle on: `conflict`, `responsiveness`, `comprehensiveness`, `no_reviewers`, `behaviour` (1–5). Shown on the reviewer dashboard with an overall average.

12. Frontend Architecture

- **Layout:** `resources/views/layouts/app.blade.php` (~1000 lines) — global shell including sidebar, top bar, CSRF meta, jQuery, Bootstrap, and shared JS handlers (e.g., the delegated `.open-grade-modal` AJAX handler, project menu toggling).
- **Styling:** Bootstrap 5 + custom CSS with design tokens (CSS variables): `--brand-500`, `--ink-*`, `--sand-50`, `--gold-500`, `--success`, `--danger`, etc.
- **Panels:** consistent `.panel`, `.panel-head`, `.panel-body` component classes.
- **Modals:** Workflow modals loaded via AJAX into a shared modal container (`WorkflowController@modal`).
- **AJAX patterns:** jQuery `$.post / $.ajax` with CSRF token; JSON responses `{success, message, error, ...}`.
- **Role-aware rendering:** Views conditionally render sections per role (e.g., reviewer sees limited project detail; admin/LPI see outcomes, reports, grading).

13. Configuration

Key config files:

- `config/database.php` — MySQL connection (default `mysql`).
- `config/filesystems.php` — local disk (default).
- `config/services.php` — external API credentials (SIS, CrossRef, Elsevier, Budget).
- `config/cors.php` — CORS settings (`api/*`).
- `config/sanctum.php` — API token behavior.
- `.env` — environment-specific settings (DB, mail, API keys, flags).

Relevant environment flags:

- `APP_DEBUG`, `APP_ENV`, `APP_KEY`, `APP_URL`
- `DB_*` — database connection
- `FILESYSTEM_DRIVER=local`
- `MAIL_*` — SMTP (defaults to `mailhog` in dev)
- `STUDENT_API_*`, `CROSSREF_API_*`, `ELSEVIER_API_*`, `BUDGET_API_*` — integrations
- `BUDGET_API MOCK`, `STUDENT_API_USE_TEST_RESPONSE` — dev mock flags

14. Development & Deployment

14.1 Local Setup

```
composer install
npm install
cp .env.example .env          # configure DB, mail, API keys
php artisan key:generate
php artisan migrate
```

```
npm run dev          # compile assets (Laravel Mix)
php artisan serve
```

14.2 Common Commands

```
php artisan migrate      # run database migrations
php artisan view:cache   # compile Blade views (verify template syntax)
php artisan config:cache
php artisan route:list   # inspect routes
npm run watch            # watch frontend assets
```

14.3 Live Deployment (XAMPP + Virtual Host)

This section provides the complete steps to deploy RTS on a **live server** running **XAMPP on Windows**. Follow the steps in order.

Server profile used in this guide: XAMPP 7.x (with PHP 7.x/8.x), Apache on Windows, MySQL. The application document root is `C:/xampp74/htdocs/LRTS`.

14.3.1 Prerequisites

- **XAMPP** (V.7.0.0 or newer) installed on the server.
- MySQL (included with XAMPP) running and a database created for the application.
- The RTS application source copied into `htdocs`, or cloned from the Git repository:
`bash cd C:/xampp74/htdocs git clone <repo-url> LRTS`

14.3.2 Setup the Virtual Host

1. Map the sub-domain to localhost (test only).

Edit the hosts file:

```
C:\Windows\System32\drivers\etc\hosts
```

Add the line:

```
127.0.0.1 lrts.qu.edu.qa
```

This line is only for **local testing** so the virtual host works on the server machine before DNS is live. It is **not** required (and should be removed on the live server) once DNS points the sub-domain to the server's public IP.

2. Enable the virtual host include in Apache.

In `C:\xampp74\apache\conf\httpd.conf`, make sure the following line is **not commented out** (remove the leading `#` if present):

```
apache Include conf/extra/httpd-vhosts.conf
```

3. Configure the virtual host.

Edit `C:\xampp74\apache\conf\extra\httpd-vhosts.conf`. The correct configuration has **two** virtual host blocks — a catch-all default (required so Apache does not

use the wrong doc root) and the RTS host. Add the following:

```
``apache
```

```
# — Catch-all default (required to avoid serving wrong doc root) —
```

```
ServerAdmin webmaster@qu.edu.qa
```

```
DocumentRoot "C:/xampp74/htdocs"
```

```
ServerName localhost
```

```
# — RTS live host (HTTP) —
```

```
ServerAdmin lrts@qu.edu.qa
```

```
DocumentRoot "C:/xampp74/htdocs/LRTS/public"
```

```
ServerName lrts.qu.edu.qa
```

```
ErrorLog "logs/lrts.qu.edu.qa-error.log"
```

```
CustomLog "logs/lrts.qu.edu.qa-access.log" common
```

```
<Directory "C:/xampp74/htdocs/LRTS/public">
  Options Indexes FollowSymLinks
  AllowOverride All
  Require all granted
</Directory>
```

```
...
```

Important corrections vs. the original draft:

- `DocumentRoot` must point to the Laravel **public** folder (`C:/xampp74/htdocs/LRTS/public`), **not** the project root. Laravel's `public/index.php` is the single entry point.
- `ServerName` should be the real sub-domain `lrts.qu.edu.qa` (not a placeholder like `dummy-host2.example.com`).
- The `<Directory>` block with `AllowOverride All` is required so the `.htaccess` file (which rewrites all requests to `index.php`) is honoured.

1. Restart Apache so the virtual host takes effect:

- Via the XAMPP Control Panel, or
- From the command line:

```
bash C:\xampp74\apache\bin\httpd.exe -k restart
```

14.3.3 DNS Settings

□ Performed by ITS (Information Technology Services, Qatar University).

The application team provides the server's IP address; ITS performs the DNS configuration and makes the sub-domain live.

1. Register the sub-domain. Contact ITS (Information Technology Services) and request that a sub-domain be created and pointed to the server's IP address:

`lrts.qu.edu.qa → <server public IP>`

ITS performs the following:

- Creates the DNS `A` record (and, for HTTPS, the `CNAME / A` record) for `lrts.qu.edu.qa`.
- Assigns the sub-domain to the server's public IP.
- (Optional) Issues the SSL certificate for HTTPS.

This step is what makes the system reachable over the network from other machines.

The application team should **coordinate with ITS** and provide the server IP; no direct server access to DNS is available to the application team.

14.3.4 Application Configuration (Laravel)

1. **Configure the `.env` file** for production:

```
```.ini
APP_ENV=production
APP_DEBUG=false
APP_URL=https://lrts.qu.edu.qa # use https once SSL is installed
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=lrts_mcp
DB_USERNAME=root
DB_PASSWORD=
```
```

Set `APP_DEBUG=false` so errors and configuration are not exposed publicly.

1. **Run migrations** to create/update the database schema:

```
bash php artisan migrate --force
```

2. **Optimize the application for production:**

```
bash php artisan config:cache php artisan route:cache php artisan view:cache php
artisan event:cache
```

3. **Set filesystem permissions** (Windows XAMPP generally handles this, but ensure the web server can write to):

- `storage/framework/` (sessions, cache, views)
- `storage/logs/`
- `storage/app/uploads/` (uploaded PDFs)

4. **Verify file permissions for storage links** — if the app uses the `public` disk, create the storage symlink:

```
bash php artisan storage:link
```

14.3.5 SSL Certificate (HTTPS)

To serve the system over **HTTPS**, obtain and install an SSL certificate.

Who does what: The **application team** generates the CSR and installs the certificate on the server. **ITS** signs/issues the certificate and returns the `.crt / .cer` file.

1. **Generate a Certificate Signing Request (CSR)** (application team). Open a command prompt in the OpenSSL directory (XAMPP ships OpenSSL) and run:

```
bash openssl req -new -newkey rsa:2048 -nodes -keyout server.key -out server.csr \
-subj "/C=QA/L=Doha/O=Qatar University/OU=IT Department/CN=*.qu.edu.qa"
```

This produces two files:

- `server.key` — the **private key** (keep this yourself, never share it).
- `server.csr` — the **certificate signing request** (send this to ITS).

2. **Submit the CSR to ITS** (performed by ITS). ITS signs the certificate and returns a certificate file (`server.cer` or `server.crt`). ITS may also issue the certificate directly if the domain is already registered under Qatar University.

3. **Install the certificate.**

- Copy `server.crt` into `C:\xampp74\apache\conf\ssl.crt\`
- Copy `server.key` into `C:\xampp74\apache\conf\ssl.key\`

4. **Add an HTTPS virtual host.** In

```
C:\xampp74\apache\conf\extra\httpd-vhosts.conf add:
```

```
``apache
```

```
# — RTS live host (HTTPS) —
```

```
ServerAdmin lrts@qu.edu.qa
```

```
DocumentRoot "C:/xampp74/htdocs/LRTS/public"
```

```
ServerName lrts.qu.edu.qa
```

```
SSLEngine on
SSLCertificateFile "C:/xampp74/apache/conf/ssl.crt/server.crt"
SSLCertificateKeyFile "C:/xampp74/apache/conf/ssl.key/server.key"

<Directory "C:/xampp74/htdocs/LRTS/public">
    Options Indexes FollowSymLinks
    AllowOverride All
    Require all granted
</Directory>
```

```
...
```

5. **Restart Apache** for the SSL changes to take effect:

```
bash C:\xampp74\apache\bin\httpd.exe -k restart
```

6. **(Recommended) Force HTTPS redirection.** Ensure all traffic uses HTTPS by adding the following to `C:\xampp74\htdocs\LRTS\public\.htaccess` (at the top):
- ```
apache RewriteEngine On RewriteCond %{HTTPS} off RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```
- > The `.htaccess` file must already contain Laravel's default rewrite rules
- > (routing all requests through `index.php`). Add only the HTTPS block above it.

### 14.3.6 Live Deployment Checklist

The table marks each task with the responsible party: **App** (application team) or **ITS** (Information Technology Services, Qatar University).

Step	Task	Performed by	Status
1	XAMPP installed (V.7.0.0+)	App	<input type="checkbox"/>
2	RTS copied/cloned into <code>htdocs/LRTS</code>	App	<input type="checkbox"/>
3	Hosts file entry (local test only)	App	<input type="checkbox"/>
4	Virtual host configured (HTTP, doc root = <code>public</code> )	App	<input type="checkbox"/>
5	Sub-domain DNS ( <code>lrts.qu.edu.qa</code> ) registered	<b>ITS</b>	<input type="checkbox"/>
6	<code>.env</code> set for production ( <code>APP_DEBUG=false</code> )	App	<input type="checkbox"/>
7	<code>php artisan migrate --force</code> run	App	<input type="checkbox"/>
8	Config/route/view caches built	App	<input type="checkbox"/>
9	Storage directories writable	App	<input type="checkbox"/>
10	CSR generated and sent to ITS	App	<input type="checkbox"/>
11	SSL certificate signed / issued	<b>ITS</b>	<input type="checkbox"/>
12	SSL certificate installed on server	App	<input type="checkbox"/>
13	HTTPS virtual host + restart Apache	App	<input type="checkbox"/>
14	HTTPS redirect in place	App	<input type="checkbox"/>

#### Summary of ITS (Information Technology Services) responsibilities:

1. Register the sub-domain `lrts.qu.edu.qa` and point it (DNS `A` / `CNAME` record) to the server's public IP.
2. Sign/issue the SSL certificate after receiving the CSR from the application team.

All other tasks (server setup, application configuration, virtual host, certificate installation, HTTPS redirection) are performed by the application team.

## 14.4 Testing

- PHPUnit configured ( `phpunit.xml` ).
- Tests under `tests/` — includes `ProjectModelTest` (unit) covering Project model helpers (status, lifecycle, actions).

---

Documentation generated from code analysis of the RTS v2 codebase.